

Chain of Responsibility (CoR)

Design Pattern

PRESENTED BY

Team returnZero

Instructor

Md Samsuddoha

Assistant Professor,
Dept. of CSE
University of Barishal



Table of Contents

1 Introduction to Chain of Responsibility

2 Intent of the Pattern

3 Why This Pattern Is Needed

4 Main Problem

5 Improved Solution

6 Key Features of CoR Pattern

7 Structure of CoR Pattern

8 Real-World Analogy : ATM Machine

9 How CoR Works Step by Step

10 Pros and Cons

11 When to Use CoR Pattern

12 When Not to Use CoR Pattern

13 ATM Cash Withdrawal Example
(Code Implementation)

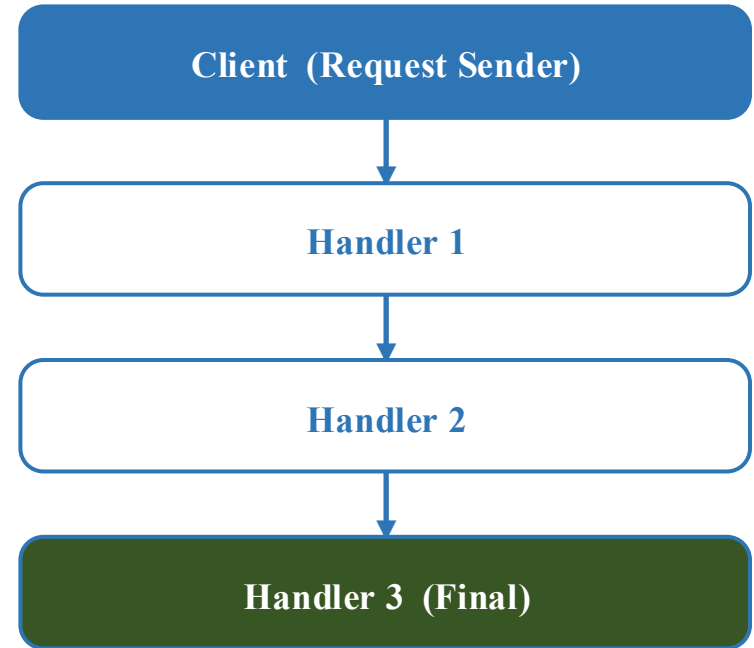
14 Coding Example: ATM Machine



Introduction to Chain of Responsibility

What is it?

- Chain of Responsibility is a behavioral design pattern.
- It passes a request along a chain of handlers.
- Each handler decides to process the request or pass it to the next handler in the chain.
- The sender does not know which handler will process the request.



Chain of Responsibility = a chain of handlers where each one tries to handle the request before passing it on.



Intent of the Pattern

Intent:

avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until an object handles it.



Decouple Sender & Receiver

The sender doesn't need to know which handler will process the request.



Centralized Control

Each handler focuses on its own responsibility only.



Flexible Chain

Handlers can be added, removed, or reordered at runtime.



Multiple Handlers

More than one handler can process parts of the same request.

"Instead of one object handling all requests, each handler in the chain tries to handle it."



Why This Pattern Is Needed

The reality of real software

- Many requests must pass through multiple checks or processing steps.
- Hard-coding all conditions into one class makes it rigid.
- Adding a new processing step means changing existing code.
- Without CoR, the sender must know all possible handlers.

Without CoR



Every handler knows about others → Tight coupling

With CoR



Clean linear chain → Loose coupling



Problem

Online Ordering System

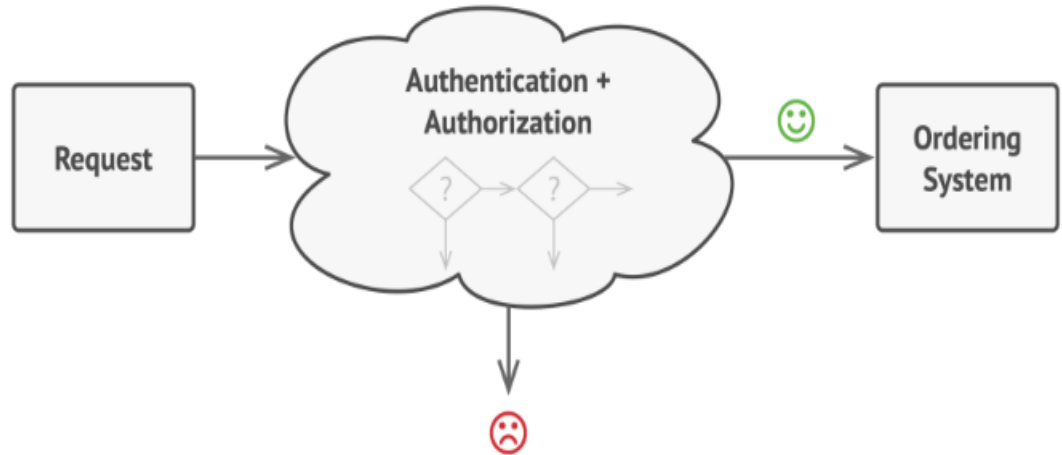
Suppose an online ordering system receives requests from users.
Before processing an order, several checks are required:

1. User Authentication
2. Authorization Check
3. Input Validation
4. Security Check
5. Cache Check

If all checks are implemented inside one class:

Problems

- Tight coupling
- Large methods
- Hard maintenance
- Difficult extension
- Difficult testing



Before Chain of Responsibility

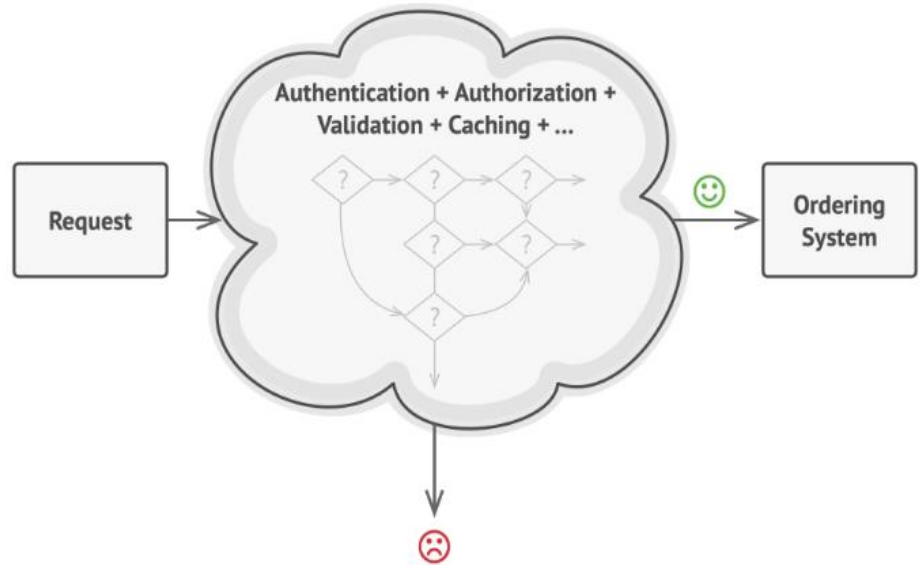
Direct and Complex Processing

All validations are placed inside one module.

Issues

- ✗ Huge conditional statements
- ✗ Difficult debugging
- ✗ Poor scalability
- ✗ Low flexibility

When a new validation is needed, exist code must be modified.



Solution

Use a Chain of Handlers

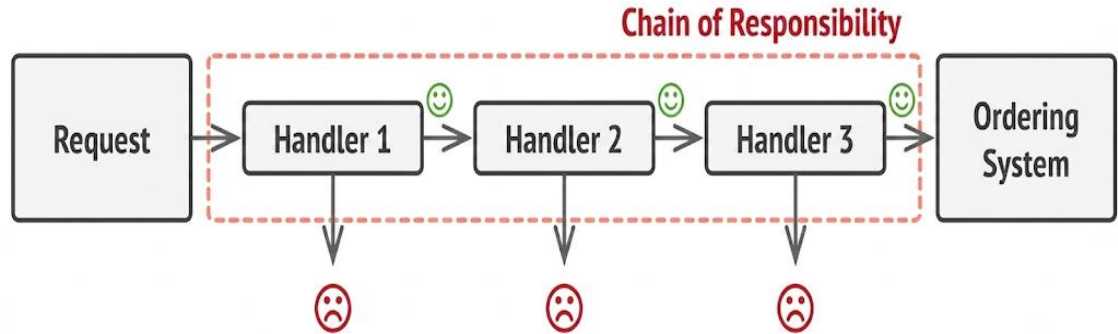
Transform each validation into a separate handler.

Examples:

- Authentication Handler
- Authorization Handler
- Validation Handler
- Cache Handler

Each handler:

1. Performs its own task
2. Passes request to next handler



Handlers are lined up one by one, forming a chain.

Flow



After Applying Chain of Responsibility

Central Idea

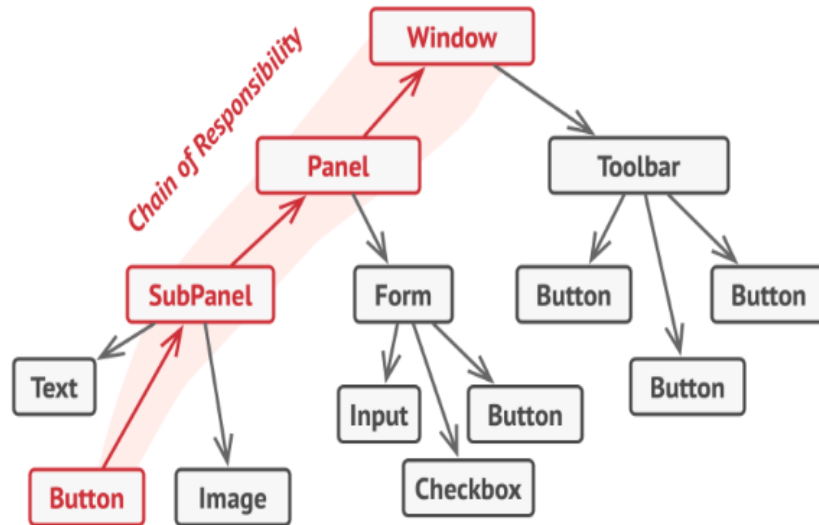
Request travels through a chain.

Each handler decides:

- Handle request
- Forward request

Benefits

- ✓ Flexible
- ✓ Easy to extend
- ✓ Easy maintenance
- ✓ Cleaner code



A chain can be formed from a branch of an object tree.



Key Features of Chain of Responsibility Pattern

1

Decoupled Handlers

Sender doesn't know which handler will process the request.

2

Dynamic Chain

Handlers can be added, removed, or reordered at runtime.

3

Single Responsibility

Each handler focuses only on its own processing logic.

4

Open/Closed Principle

New handlers can be added without changing existing ones.

5

Flexible Termination

Any handler can stop the chain by handling the request.

6

Pass-Through Support

A handler can pass the request without processing it.



Structure of Chain of Responsibility Pattern

1. Handler Interface

Declares the `handle(request)` method. May also declare a `setNext(handler)` method to build the chain.

2. Abstract Handler

Implements the default chaining behavior — if it cannot handle, it passes to the next handler.

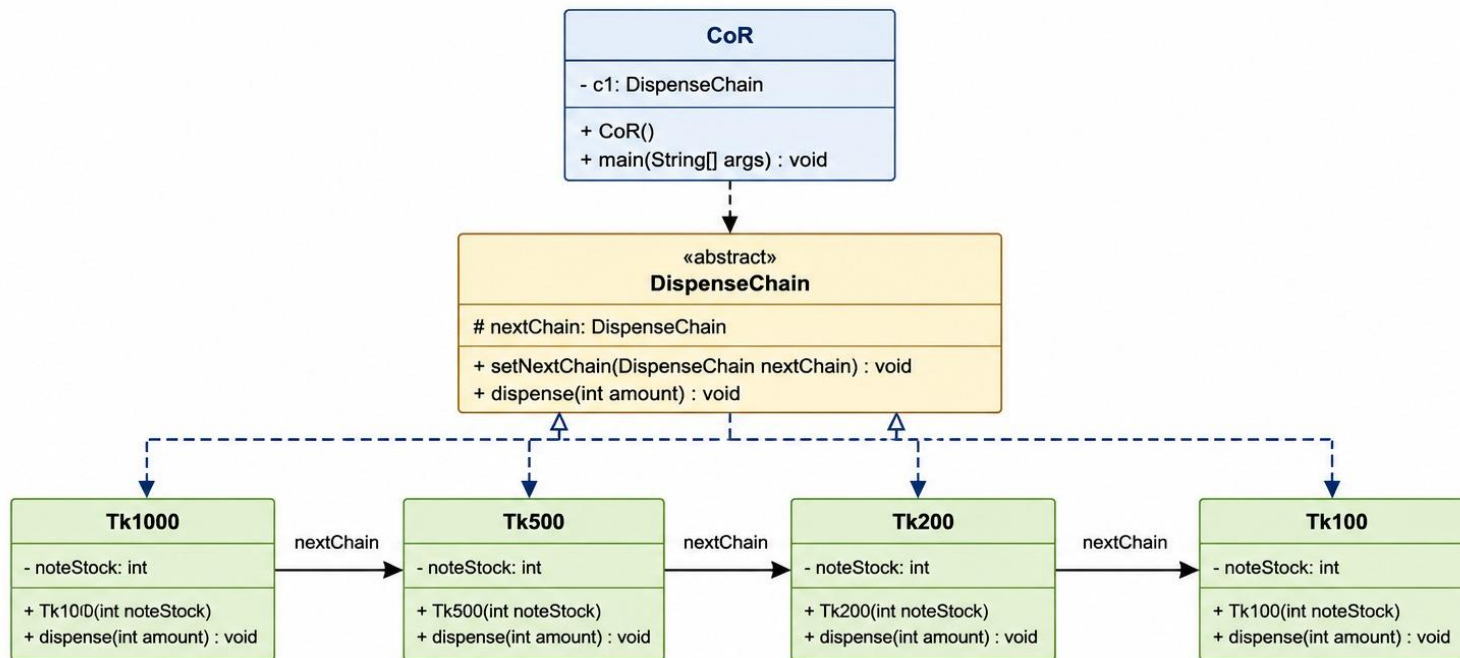
3. Concrete Handlers

Each one processes requests it is responsible for. Otherwise, it forwards the request to the next.

4. Client

Composes the chain and sends the request only to the first handler in the chain.

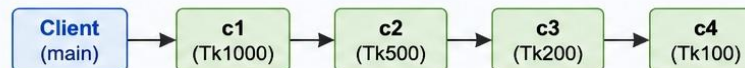
UML Structure Diagram According to Real-World Analogy : ATM Machine



Chain Configuration in CoR()

- `c1 = new Tk1000(100)`
 - `c2 = new Tk500(50)`
 - `c3 = new Tk200(20)`
 - `c4 = new Tk100(10)`
- `c1.setNextChain(c2)`
 - `c2.setNextChain(c3)`
 - `c3.setNextChain(c4)`

Runtime Flow



Tk1000 → Tk500 → Tk200 → Tk100



Real-World Analogy : ATM Machine



Result: $2 \times ₹1000 + 1 \times ₹200 + 1 \times ₹100 = ₹2,300$ dispensed successfully!

- As long as ₹1000 notes are available, the ₹1000 Handler dispenses them first.
- If a denomination cannot fully cover the remainder, that handler passes it to the next.
- The chain continues until the full amount is dispensed or funds are insufficient.

How CoR Works Step by Step

1 Client creates the chain by linking handlers: `H1.setNext(H2); H2.setNext(H3);`

2 Client sends the request only to the first handler in the chain.

3 Handler 1 checks if it can process the request. If yes, it handles it and the chain stops.

4 If Handler 1 cannot process it, it calls `next.handle(request)` to forward it to Handler 2.

5 This continues until a handler processes the request or the chain ends.

6 If no handler processes it, the request may be ignored or a default action is taken.

The chain gives every handler a chance to act — and stops as soon as one does.



Pros and Cons



Pros

- Reduces direct dependencies between sender and receivers
- New handlers can be added without changing existing code
- Chain order is flexible — easy to reconfigure
- Each handler has a single, clear responsibility
- Cleaner and more maintainable code structure
- Supports runtime composition of chains



Cons

- Request may go unhandled if no handler matches
- Long chains can be hard to debug and trace
- Performance may suffer if the chain is very long
- Chain order errors can cause incorrect behavior



When to Use Chain of Responsibility Pattern

Use it when...

- More than one object may handle a request
- The handler isn't known at compile time
- You want to issue a request to one of several objects without specifying the receiver explicitly
- You want to add or change handling behavior without modifying existing classes
- Request processing steps form a natural ordered pipeline

Typical examples

- ATM cash dispensing machine
- Customer support ticket system
- HTTP request middleware pipeline
- Event propagation in UI (click bubbling)
- Logging levels (DEBUG → INFO → ERROR)
- Access control / permission checks



When Not to Use CoR Pattern

X Only One Handler Needed

If there is only one possible handler for the request, CoR adds unnecessary complexity. Use a simple method call instead.

X Every Request Must Be Handled

If all requests must be guaranteed to be processed, CoR's pass-through behavior may leave some unhandled.

X Very Simple Systems

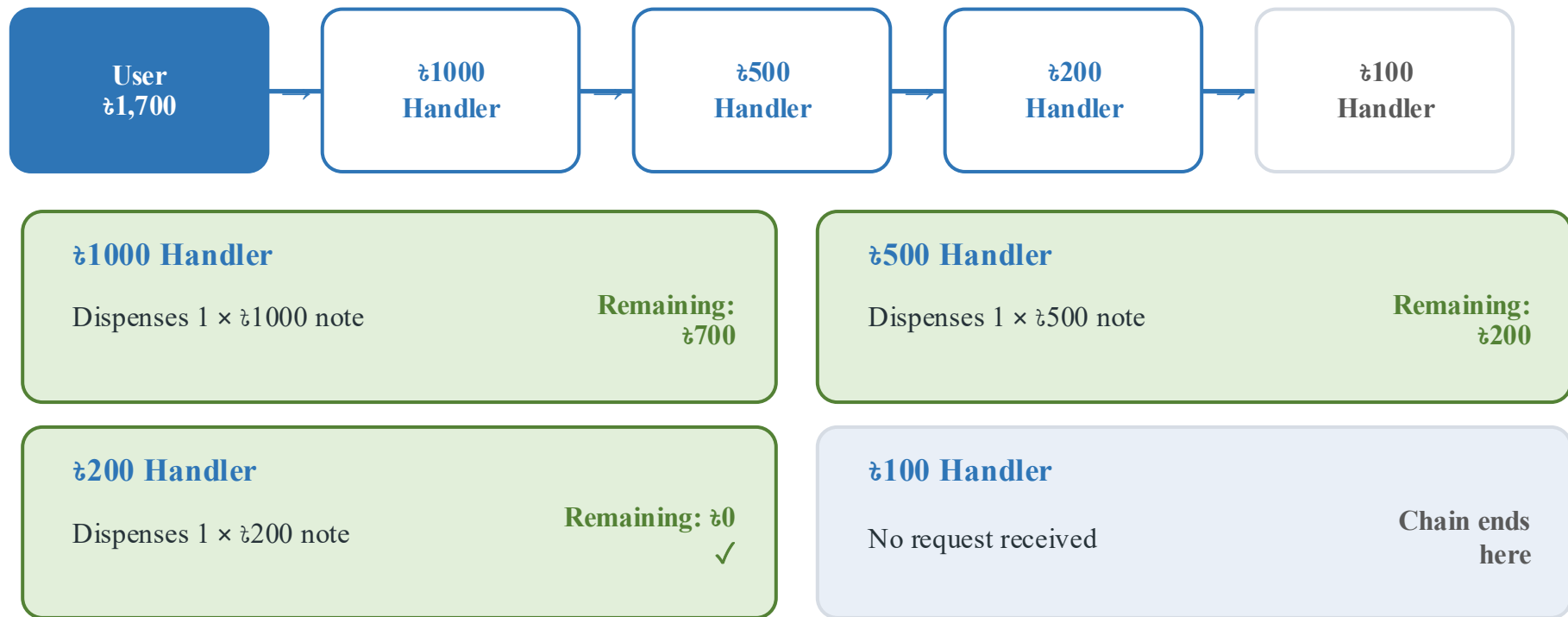
For small or simple systems, the overhead of creating a chain of handlers is not justified.

X Performance-Critical Paths

Long chains introduce latency. If every millisecond counts, avoid passing through many handlers.



Coding Example : ATM Machine Withdrawal System



ATM Handlers = Chain of Responsibility Notes = Handled Requests User = Client

```
import java.util.Scanner;

//Abstract class for DispenseChain
abstract class DispenseChain{
    protected DispenseChain nextChain;

    public void setNextChain(DispenseChain nextChain){
        this.nextChain = nextChain;
    }

    public abstract void dispense(int amount);
}
```

```
//For 1000 Tk Notes
class Tk1000 extends DispenseChain{
    private int noteStock;

    public Tk1000(int noteStock){
        this.noteStock = noteStock;
    }

    public void dispense(int amount){
        if(amount >= 1000 && this.noteStock > 0){
            int needNote = amount / 1000;
            int dispenseNote = Math.min(needNote, this.noteStock);

            this.noteStock -= dispenseNote;

            int dispenseAmount = dispenseNote * 1000;
            int remainder = amount - dispenseAmount;

            System.out.println("Dispensing " + dispenseNote +
                " notes of 1000 Tk. (Remaining Stock: "
                + this.noteStock + ")");
        }
    }
}
```

```
        if(remainder > 0 && this.nextChain != null){
            this.nextChain.dispense(remainder);
        }
    }
    else if(this.nextChain != null){
        this.nextChain.dispense(amount);
    }
    else{
        System.out.println("Error : Cannot dispense the remaining "
            + amount + " Tk.");
    }
}
}
```



```
//For 200 Tk Notes
class Tk200 extends DispenseChain{
    private int noteStock;

    public Tk200(int noteStock){
        this.noteStock = noteStock;
    }

    public void dispense(int amount){
        if(amount >= 200 && this.noteStock > 0){
            int needNote = amount / 200;
            int dispenseNote = Math.min(needNote, this.noteStock);

            this.noteStock -= dispenseNote;

            int dispenseAmount = dispenseNote * 200;
            int remainder = amount - dispenseAmount;
```

```
System.out.println("Dispensing " + dispenseNote +
    " notes of 200 Tk. (Remaining Stock: "
    + this.noteStock + ")");

    if(remainder > 0 && this.nextChain != null){
        this.nextChain.dispense(remainder);
    }
}
else if(this.nextChain != null){
    this.nextChain.dispense(amount);
}
else{
    System.out.println("Error : Cannot dispense the remaining "
        + amount + " Tk.");
}
}
```

```
//For 100 Tk Notes
class Tk100 extends DispenseChain{
    private int noteStock;

    public Tk100(int noteStock){
        this.noteStock = noteStock;
    }

    public void dispense(int amount){
        if(amount >= 100 && this.noteStock > 0){
            int needNote = amount / 100;
            int dispenseNote = Math.min(needNote, this.noteStock);

            this.noteStock -= dispenseNote;

            int dispenseAmount = dispenseNote * 100;
            int remainder = amount - dispenseAmount;

            System.out.println("Dispensing " + dispenseNote +
                               " notes of 100 Tk. (Remaining Stock: "
                               + this.noteStock + ")");
        }
    }
}
```

```

        if(remainder > 0){
            System.out.println("Error : Machine ran out of cash! Remaining "
                               + remainder + " Tk cannot be dispensed.");
        }
    }
    else{
        System.out.println("Error : Machine ran out of 100 Tk notes or amount is too small. Remaining "
                           + amount + " Tk.");
    }
}
}
}

```

//This the Main class

```

public class CoR{

    private DispenseChain c1;

    public CoR(){

        //Set every note stock by using constructor
        this.c1 = new Tk1000(100);
        DispenseChain c2 = new Tk500(50);
        DispenseChain c3 = new Tk200(20);
        DispenseChain c4 = new Tk100(10);
    }
}

```



```
//Connect to the Chain
c1.setNextChain(c2);
c2.setNextChain(c3);
c3.setNextChain(c4);
}

public static void main(String[] args){

    CoR atm = new CoR();

    Scanner sc = new Scanner(System.in);

    //Take the withdraw amount from the user
    System.out.print("Enter your withdraw amount : ");
    int withdrawAmount = sc.nextInt();
```

```
System.out.println("Withdrawing : " + withdrawAmount + " Tk.");

if(withdrawAmount % 100 != 0){
    System.out.println("Error : Amount should be in multiples of 100.");
    return;
}

atm.c1.dispense(withdrawAmount);
}
}
```

References

Refactoring Guru

<https://refactoring.guru/design-patterns/chain-of-responsibility>

Gang of Four (GoF)

Design Patterns: Elements of Reusable Object-Oriented Software

Java Documentation

<https://docs.oracle.com>

Video Tutorial

<https://youtu.be/Fc6cTDJ6Q-c>

Conclusion

Chain of Responsibility is a behavioral design pattern that allows a request to pass through a sequence of handlers until it is processed.

By separating responsibilities into individual handlers, the pattern:

- i. Reduces coupling between sender and receiver
- ii. Improves flexibility and maintainability
- iii. Makes the system easier to extend
- iv. Promotes cleaner and more organized code

This pattern is especially useful when multiple processing steps or validations need to be performed in a specific order.

Thank You!

Chain of Responsibility Design Pattern

Team returnZero | Dept. of CSE, University of Barishal

Any Questions?